

Hash Cracking with John the Ripper (JtR)

1. What Are Hashes?

- A **hash** is a fixed-length string generated from input data (like a password) using algorithms such as MD5, SHA-256, or SHA-512.
- Hashes are **one-way functions** — they can't be easily reversed, but can be cracked using **brute-force or wordlists**.

Supported Hashes in JtR

Hash Type Format Name in JtR Length (bits) Common Use

MD5	raw-md5	128-bit	Old systems, weak hashing
SHA-256	raw-sha256	256-bit	Stronger, secure hashing
SHA-512	raw-sha512	512-bit	Stronger, used in Linux shadow

Step-by-Step Cracking Examples

◆ A. Cracking MD5 Hash

1. Generate MD5 Hash:

```
echo -n "admin123" | md5sum
```

Output:

```
0192023a7bbd73250516f069df18b500 -
```

Save it in a file:

```
echo "0192023a7bbd73250516f069df18b500" > md5.txt
```

2. Crack Using JtR:

```
john --format=raw-md5 md5.txt
```

◆ B. Cracking SHA-256 Hash

✓ 1. Generate SHA-256:

```
echo -n "pineapple" | sha256sum
```

Output:

```
d5e6c3c7114be19d9a8a490f1c7de9c7cbf857bc5cd95a46c389205e9fce2ab8
```

Save it:

```
echo
```

```
"d5e6c3c7114be19d9a8a490f1c7de9c7cbf857bc5cd95a46c389205e9fce2ab8" >  
sha256.txt
```

✓ 2. Crack Using JtR:

```
john --format=raw-sha256 sha256.txt
```

◆ C. Cracking SHA-512 Hash

✓ 1. Generate SHA-512:

```
echo -n "password123" | sha512sum
```

Output:

```
Ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f5a1d  
6c9731f213b0c54a539b5f0f3b3638a1310e2076b649851bc7e8fbe6f5fb
```

Save to file:

```
echo
```

```
"ef92b778bafe771e89245b89ecbc08a44a4e166c06659911881f383d4473e94f5a1  
d6c9731f213b0c54a539b5f0f3b3638a1310e2076b649851bc7e8fbe6f5fb" >  
sha512.txt
```

✅ 2. Crack Using JtR:

```
john --format=raw-sha512 sha512.txt
```

🔄 Optional: Use Wordlist

To use a custom or default wordlist like rockyou.txt:

```
john --format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt md5.txt
```

📁 1. rockyou.txt – Most famous wordlist

Location in Kali Linux:

```
/usr/share/wordlists/rockyou.txt
```

To use it:

```
gunzip /usr/share/wordlists/rockyou.txt.gz
```

✅ Check Cracked Passwords:

```
john --show md5.txt
```

📌 Important Notes:

- Use --format to specify correct hash type.
- Always remove the trailing - from the hash if present.
- Use --wordlist to make cracking faster if password is weak.
- SHA-256 and SHA-512 take longer than MD5 due to stronger encryption.

🧠 Summary Table

Hash Type Sample Command

MD5 john --format=raw-md5 md5.txt

SHA-256 john --format=raw-sha256 sha256.txt

SHA-512 john --format=raw-sha512 sha512.txt

◆ 1. Single Mode

✅ What is it?

Single mode is a *highly targeted attack* that uses information from user-related fields like:

- Usernames
- GECOS (Full names, job titles, etc.)
- Login IDs

It's extremely useful when you're working with **/etc/passwd** and **/etc/shadow** files, or password files with username and hash pairs.

📘 Example Use Case:

If a user is named "**john.doe**", JtR will try smart combinations like:

- john
- johndoe
- doejohn
- john123
- JohnDoe1

🧠 Why is it called “single”?

Because it's designed to **target one user at a time**, using their personal info to guess their password.

🔧 Command:

```
john --single passwd_hash.txt
```

💡 Make sure the file is in the format:

```
username:hash
```

Otherwise, it won't trigger the smart guessing logic.

🔍 Behind the scenes:

Single mode uses built-in rules under [List.Rules:Single] from john.conf.

It generates variants like:

- lowercase
- capitalized
- reversed
- number suffixes

◆ 2. Incremental Mode

✓ What is it?

Incremental mode is a **brute-force** attack mode that tries **every possible combination of characters**, starting from short to long.

🔥 Very Powerful — But Slow

- Tries all characters (letters, numbers, symbols)
- Smart order based on character frequency and patterns
- Can take hours/days depending on length and CPU

🔧 Command:

```
john --incremental hash.txt
```

By default, this uses:

```
[List.Rules:Incremental:All]
```

But you can specify modes like:

- --incremental=ASCII → All ASCII chars
- --incremental=Alpha → Letters only
- --incremental=Digits → Numbers only

🕒 Example Time:

Cracking a 5-letter lowercase password:

```
john --incremental=Lowercase hash.txt
```

🧠 Internally, it generates:

- a, b, c...z
- aa, ab...zz
- aaa, aab...zzz

Comparison: Single vs Incremental

Feature	Single Mode	Incremental Mode
Type	Smart, user-specific guesses	Full brute-force
Speed	Fast	Slow
Accuracy	High (if user info available)	Guaranteed (eventually)
Best for	/etc/passwd style cracking	Unknown/random password cracking
Customizable?	Yes ([List.Rules:Single])	Yes (--incremental=<mode>)

Example Test

If you want to test on a file:

```
echo -n "12345" | md5sum > hash.txt
```

Then run:

```
john --format=raw-md5 hash.txt --incremental=Digits
```

JtR will eventually find 12345.